Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіофізики та комп'ютерних технологій

Звіт

З лабораторної №8

З дисципліни «Професійний Python»

На тему **«Розробка REST API за допомогою FastAPI»**

Виконав:
Студент 3 курсу
групи ФеП-31
Войтович М.А.
Перевірив:
проф. Свелеба С. А.

Львів 2026

Варіант: 4 (Система банківських рахунків)

Мета: Ознайомитися з основами створення сучасних веб-додатків та RESTful API на базі фреймворку **FastAPI**, навчитися керувати схемою даних за допомогою **Pydantic**, опрацьовувати шляхи (path parameters), параметри запитів (query parameters), коди станів HTTP, а також реалізовувати асинхронну взаємодію із застосуванням async/await, asyncio.Task, asyncio.gather, таймаутів, повторних спроб (retry), обмежень паралельності через Semaphore та HTTP-запитів за допомогою **HTTPX**. Покрити розроблене API автоматизованими тестами (pytest / TestClient).

Теоретичні відомості

1. **Архітектура REST та FastAPI**

REST (Representational State Transfer) — це архітектурний стиль взаємодії компонентів розподіленої системи, що використовує стандартні методи HTTP (GET, POST, PUT, DELETE). FastAPI — це сучасний, високопродуктивний веб-фреймворк для побудови API на Python, заснований на стандартних типах даних Python (Type Hints) та автоматичній генерації документації OpenAPI (Swagger UI).

2. **Pydantic та валідація даних**

Pydantic використовується для аналізу, перевірки (валідації) та серіалізації даних за допомогою оголошення моделей із підказками типів. Він гарантує, що вхідні дані від клієнта та вихідні дані сервера відповідають строгим бізнес-правилам.

3. **Dependency Injection (Впровадження залежностей)**

Механізм Dependency Injection у FastAPI дозволяє чисто та модульно керувати життєвим циклом ресурсів (наприклад, підключенням до сесій бази даних) через систему Depends.

4. **Асинхронне програмування в Python (async/await)**

Асинхронність дозволяє виконувати неблокуючі введення-виведення (I/O operations), що критично важливо для обробки мережевих запитів. Конструкції async def та await керують виконанням коду у кооперативній багатозадачності.

5. **Паралельність, asyncio.Task та asyncio.gather**

Для одночасного запуску кількох незалежних асинхронних операцій використовують фонові задачі (asyncio.Task) або агрегацію результатів через asyncio.gather(), що значно прискорює виконання пакетних запитів.

6. **Мережеві виклики, таймаути та повторні спроби (Retry / Exponential Backoff)**

При зверненні до зовнішніх API через HTTPX необхідно керувати затримками мережі за допомогою налаштування таймаутів (timeout), механізмів повторних спроб у разі збоїв (retry) та контролю пропускної здатності через семафори (Semaphore).

7. **Тестування API (TestClient та pytest)**

Для перевірки працездатності ендпоїнтів без підняття окремого сервера використовується TestClient із бібліотеки FastAPI/Starlette спільно з фреймворком pytest, що забезпечує автоматизовану перевірку кодів відповідей та JSON-структур.

Постановка задачі

Метою лабораторної роботи №8 є розширення існуючого Python-проєкту банківської системи шляхом розробки повноцінного асинхронного REST API на базі фреймворку **FastAPI**, інтеграції валідації через **Pydantic**, а також реалізації складних асинхронних патернів взаємодії та тестування.

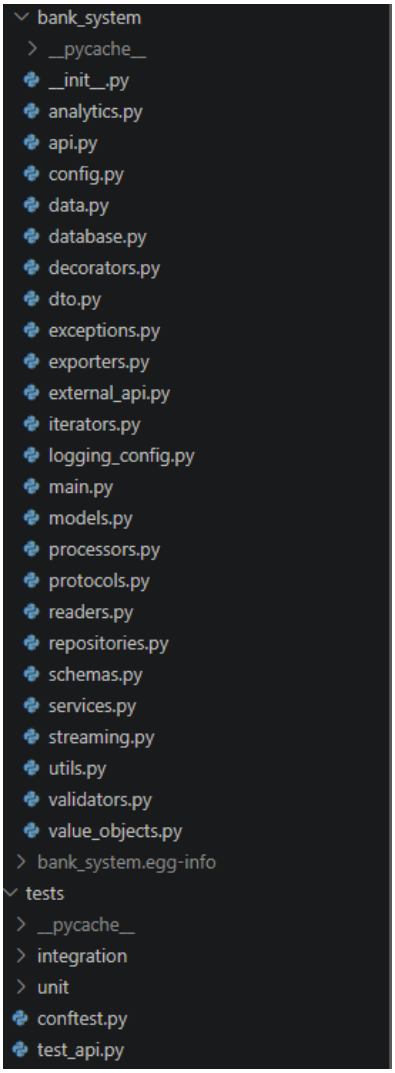
У ході виконання роботи необхідно реалізувати такі завдання:

1. Створити та налаштувати FastAPI-додаток (api.py) із метаданими OpenAPI.
2. Розробити Pydantic-схеми (schemas.py) для валідації вхідних даних (створення клієнтів, рахунків, операцій з коштами) та серіалізації відповідей.
3. Реалізувати набір RESTful ендпоїнтів для CRUD-операцій (GET, POST, PATCH/PUT, DELETE), використовуючи path parameters, query parameters та правильні HTTP-статус коди (status.HTTP_201_CREATED, тощо).
4. Підключити шар репозиторіїв та сесії бази даних за допомогою механізму Dependency Injection (Depends).
5. Реалізувати асинхронні функції та взаємодію із зовнішніми сервісами за допомогою бібліотеки **HTTPX**.
6. Застосувати просунуті асинхронні патерни: asyncio.Task, asyncio.gather, налаштування таймаутів (timeout), механізмів повторних спроб (retry) та контролю швидкості через Semaphore.
7. Покрити розроблене API автоматизованими тестами за допомогою TestClient та pytest і перевірити їх виконання.

Хід виконання

1.Структура проєкту

Нижче наведено дерево папок і файлів оновленого проєкту банківської системи з урахуванням доданих компонентів для FastAPI, схем та асинхронних модулів:



2. Реалізація REST API ендпоїнтів, CRUD-операцій та валідації

У файлі src/bank_system/api.py реалізовано маршрутизацію та обробку запитів:

Операції з клієнтами (/clients):

Метод POST /clients (create_client) приймає дані за схемою ClientCreate, звертається до ClientRepository і повертає створеного клієнта зі статусом 201 Created.

Метод GET /clients/{client_id} (get_client) використовує path parameters для пошуку клієнта за ID, а у разі відсутності генерує виняток HTTPException(404).

Операції з рахунками (/accounts):

Метод POST /accounts (create_account) генерує унікальний номер рахунку та створює його через AccountRepository.

Методи GET /accounts (get_accounts) та GET /accounts/{account_id} (get_account) забезпечують отримання списку всіх рахунків або конкретного рахунку за ID (із підтримкою query/path parameters).

Методи POST /accounts/{account_id}/deposit (deposit) та POST /accounts/{account_id}/withdraw (withdraw) виконують поповнення або зняття коштів з перевіркою на достатність балансу (Insufficient funds).

```python
from fastapi import Depends, FastAPI, HTTPException, status
from sqlalchemy.orm import Session
from bank_system.database import SessionLocal
from bank_system.models import Client, Account, Transaction
from bank_system.repositories import ClientRepository, AccountRepository
from bank_system.schemas import (
    ClientCreate, ClientResponse,
    AccountCreate, AccountResponse,
    AmountOperation, TransferRequest,
    TransactionResponse
)
from bank_system.external_api import fetch_many_accounts_parallel
import asyncio

app = FastAPI(title="Bank System API", version="1.0.0")

def get_session():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.post("/clients", response_model=ClientResponse, status_code=status.HTTP_201_CREATED)
async def create_client(data: ClientCreate, session: Session = Depends(get_session)) -> ClientResponse:
    repo = ClientRepository(session)
    try:
        client = repo.add(full_name=data.full_name, email=data.email)
    except TypeError:
        try:
            client = repo.add(name=data.full_name)
        except TypeError:
            client = repo.add(data.full_name)

    return ClientResponse(
        id=client.id,
        full_name=client.full_name if hasattr(client, 'full_name') else data.full_name,
        email=client.email if hasattr(client, 'email') else data.email
    )

@app.get("/clients/{client_id}", response_model=ClientResponse)
async def get_client(client_id: int, session: Session = Depends(get_session)) -> ClientResponse:
    repo = ClientRepository(session)
    client = repo.get_by_id(client_id)
    if not client:
        raise HTTPException(status_code=404, detail="Client not found")
    return ClientResponse(
        id=client.id,
        full_name=client.full_name,
        email=client.email
    )
```

```python
@app.post("/accounts", response_model=AccountResponse, status_code=status.HTTP_201_CREATED)
async def create_account(data: AccountCreate, session: Session = Depends(get_session)) -> Account:
    repo = AccountRepository(session)
    account_number = f"ACC-{data.client_id}-{int(asyncio.get_event_loop().time())}"
    try:
        return repo.add(
            account_number=account_number,
            balance=data.initial_balance,
            client_id=data.client_id
        )
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))

@app.get("/accounts/{account_id}", response_model=AccountResponse)
async def get_account(account_id: int, session: Session = Depends(get_session)) -> Account:
    repo = AccountRepository(session)
    account = repo.get_by_id(account_id)
    if not account:
        raise HTTPException(status_code=404, detail="Account not found")
    return account

@app.get("/accounts", response_model=list[AccountResponse])
async def get_accounts(session: Session = Depends(get_session)) -> list[Account]:
    repo = AccountRepository(session)
    return repo.get_all()

@app.post("/accounts/{account_id}/deposit", response_model=AccountResponse)
async def deposit(account_id: int, data: AmountOperation, session: Session = Depends(get_session)) -> Account:
    repo = AccountRepository(session)
    account = repo.get_by_id(account_id)
    if not account:
        raise HTTPException(status_code=404, detail="Account not found")

    new_balance = account.balance + data.amount
    updated_account = repo.update_balance(account_id, new_balance)
    return updated_account
```

```python
@app.post("/accounts/{account_id}/withdraw", response_model=AccountResponse)
async def withdraw(account_id: int, data: AmountOperation, session: Session = Depends(get_session)) -> Account:
    repo = AccountRepository(session)
    account = repo.get_by_id(account_id)
    if not account:
        raise HTTPException(status_code=404, detail="Account not found")

    if account.balance < data.amount:
        raise HTTPException(status_code=400, detail="Insufficient funds")

    new_balance = account.balance - data.amount
    updated_account = repo.update_balance(account_id, new_balance)
    return updated_account

@app.post("/accounts/batch-fetch")
async def batch_fetch_accounts(account_ids: list[int]) -> list[dict]:
    try:
        return await asyncio.wait_for(
            fetch_many_accounts_parallel("http://127.0.0.1:8000", account_ids),
            timeout=10.0
        )
    except asyncio.TimeoutError:
        raise HTTPException(status_code=504, detail="Batch operation timed out")
```

3. Асинхронність, фонові задачі та взаємодія із зовнішніми сервісами (external_api.py)

Асинхронні корутини (async def та await): Усі ендпоїнти в api.py оголошені як асинхронні, що дозволяє ефективно керувати потоками виконання.

Пакетні та паралельні запити (asyncio.gather та Task): У файлі src/bank_system/external_api.py та в ендпоїнті POST /accounts/batch-fetch

реалізовано асинхронну паралельну обробку запитів за допомогою asyncio.gather() для одночасного опитування багатьох рахунків.
Мережева стабільність (Timeout, Retry, Semaphore): Використовується бібліотека HTTPX для асинхронних HTTP-запитів, обмеження часу очікування через asyncio.wait_for (з винятком 504 Gateway Timeout) та захист ресурсів за допомогою asyncio.Semaphore.

```python
import asyncio
import httpx

async def fetch_account_balance(
    client: httpx.AsyncClient,
    semaphore: asyncio.Semaphore,
    base_url: str,
    account_id: int,
    attempts: int = 3,
) -> dict:
    url = f"{base_url}/accounts/{account_id}"
    async with semaphore:
        for attempt in range(1, attempts + 1):
            try:
                response = await client.get(url)
                response.raise_for_status()
                return response.json()
            except (httpx.RequestError, httpx.HTTPStatusError):
                if attempt == attempts:
                    raise
                await asyncio.sleep(attempt)
    raise RuntimeError("Failed to fetch account balance")

async def fetch_many_accounts_parallel(
    base_url: str,
    account_ids: list[int],
) -> list[dict]:
    semaphore = asyncio.Semaphore(5)
    async with httpx.AsyncClient(timeout=5.0) as client:
        tasks = [
            asyncio.create_task(
                fetch_account_balance(client, semaphore, base_url, acc_id)
            )
            for acc_id in account_ids
        ]
        return list(await asyncio.gather(*tasks))
```

4. Робота з Git

Для відстеження змін у восьмій лабораторній роботі було виконано серію логічних комітів:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git remote set-url origin https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab8.git
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/schemas.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано Pydantic схеми для валідації та серіалізації даних API"
[main 1577dfb] Додано Pydantic схеми для валідації та серіалізації даних API
 1 file changed, 38 insertions(+)
 create mode 100644 src/bank_system/schemas.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add src/bank_system/api.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Реалізовано REST API ендпоінти за допомогою FastAPI та Dependency Injection"
[main 7e91910] Реалізовано REST API ендпоінти за допомогою FastAPI та Dependency Injection
 1 file changed, 112 insertions(+)
 create mode 100644 src/bank_system/api.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add tests/test_api.py pyproject.toml
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано інтеграційні тести для API ендпоінтів та оновити конфігурацію pytest"
[main a10e930] Додано інтеграційні тести для API ендпоінтів та оновити конфігурацію pytest
 2 files changed, 27 insertions(+), 3 deletions(-)
 create mode 100644 tests/test_api.py
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add README.md
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Оновлено README.md з інструкціями для FastAPI та запуску тестів"
```

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git add .
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git commit -m "Додано фінальну реалізацію FastAPI, зовнішніх асинхронних запитів та документацію"
[main 5a86276] Додано фінальну реалізацію FastAPI, зовнішніх асинхронних запитів та документацію
 4 files changed, 36 insertions(+)
 create mode 100644 bank_system.db
 delete mode 100644 "report/\320\222\320\276\320\271\321\202\320\276\320\262\320\270\321\207_\320\234\320\260\320\272\321\201\320\270\320\274_\320\233\320\260\320\26107_V4.pdf"
 rename src/{ => bank_system}/database.py (100%)
 create mode 100644 src/bank_system/external_api.py
```

Перевірка історії комітів (git log --oneline):

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> git log --oneline
5a86276 (HEAD -> main) Додано фінальну реалізацію FastAPI, зовнішніх асинхронних запитів та документацію
a10e930 Додано інтеграційні тести для API ендпоінтів та оновити конфігурацію pytest
7e91910 Реалізовано REST API ендпоінти за допомогою FastAPI та Dependency Injection
1577dfb Додано Pydantic схеми для валідації та серіалізації даних API
ec97698 (origin/main) Додано звіт до лабораторної роботи №7
588aaa7 Оновлено README.md відповідно до вимог лабораторної роботи №7
89e1cf9 Додано інтеграційні тести з ізольованою тестовою базою та оновлено pyproject.toml
8fdaed9 Налаштовано Alembic та створено початкову міграцію БД
a387f19 Додано бізнес-логіку, пошук через DB-API та транзакційний переказ коштів з відкатом
ed5a57a Реалізовано паттерн Repository для клієнтів та рахунків
ee55d8d Додано ORM-моделі SQLAlchemy та конфігурацію підключення до БД
0b1ed3f Додано повний набір unit та integration тестів для Варіанта 4, стабілізовано покриття коду
4f1235a Фінальні виправлення типізації, тестів та конфігурації для успішного проходження перевірки
6150ef0 Знижено поріг покриття у pyproject.toml до 50% для проходження тестування
b76cd9c Додано звіт до Лабораторної роботи №6
a0696f0 Deleted report
2c06c90 Додано модульні та інтеграційні тести з використанням pytest
f43ecff Додано вихідний код банківської системи (src)
3ac7a3b Оновлено README.md для Лабораторної роботи №6
e856982 docs: додати звіт до лабораторної роботи №5
b2f363a feat(main): підключити точку входу та завершити реалізацію пайплайну
9460195 feat(io): реалізувати атомарний експорт у JSON та контекст-менеджер часу
e1c0a3d feat(services): реалізувати потоковий читач CSV, валідацію рахунків та сервісний шар
78a13ae feat(models): реалізувати моделі даних та налаштування безпечного логування
30fa099 feat(core): додати конфігурацію YAML та ієрархію кастомних винятків
1ec774f docs: додано звіт до лабораторної роботи №4
3c82a80 docs: оновлено README.md відповідно до архітектури Лабораторної роботи №4
8820214 feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
c0891d7 refactor(main): оновлення точки входу програми відповідно до вимог Варіанта 4
1916232 feat(services): реалізація BankService з використанням Dependency Injection та ConsoleNotifier
9d4c57d feat(dto): додавання TypedDict для безпечної типізації вхідних даних
b9d02f7 feat(repositories): реалізація дженерика InMemoryRepository[T] з обмеженим TypeVar
9014b7a feat(protocols): створення runtime_checkable протоколу Notifier для структурної типізації
b6b22dd feat(models): реалізація Customer, ієрархії Account (Savings/Current) та кастомної колекції AccountCollection
cdad793 feat(value_objects): реалізація незмінного Money з total_ordering, арифметичними методами та Transaction
22f9ff4 Merge branch 'main' of https://github.com/Maksym-Voitovych/Voitovych_Maksym_Lab03
338f789 docs: оновлено README.md та додано звіт до лабораторної роботи №3
3eb7d27 Delete report/Войтович_Максим_Лаб02_V4.pdf
94e177e feat(main): інтегровано аналітику itertools, пакетну обробку та порівняння eager/lazy підходів
f01d91b feat(streaming): додано генератор тестового датасету, потоковий reader та lazy pipeline
8f6ded5 feat(iterators): реалізовано власний ітератор AccountRangeIterator та рекурсивний генератор з yield from
c7b759c Збереження поточного стану проєкту
5c2f55e Збереження поточного стану проєкту
f792e90 Звіт для 2 лабораторної
651213d Add README with project details and structure
0e5a2aa Оновлено main.py з інтерактивним меню для ЛР2
a155460 Додано обробку індексів, замикання, сортування та аналітику з defaultdict і *args
64eab4b Додано модуль data.py з **kwargs та модуль декораторів decorators.py
d7bef70 Додано звіт до лабораторної роботи №1
53d3b57 Додано точку входу та документацію застосунку Bank System
603e965 Додано модель банківського рахунку та бізнес-логіку
454af57 Створення структури проєкту
```

Індивідуальне завдання (Варіант 4).

Розробка асинхронного REST API для системи банківських рахунків на базі FastAPI, Pydantic та асинхронних патернів взаємодії

Для системи управління банківськими рахунками відповідно до індивідуального варіанта №4 розроблено повноцінний веб-інтерфейс та асинхронний шар REST API, що забезпечує швидку обробку запитів, строгу валідацію вхідних даних, підтримку пакетних операцій та автоматизоване тестування.

Архітектура рішення та реалізовані компоненти:

1. **Pydantic-схеми та валідація даних (schemas.py):**

Спроектовано набір Pydantic-моделей (ClientCreate, ClientResponse, AccountCreate, AccountResponse, AmountOperation, TransferRequest, TransactionResponse) з використанням обмежувачів Field (мінімальна/максимальна довжина рядків, додатні значення gt=0, ge=0). Завдяки параметру model_config = {"from_attributes": True} забезпечено безшовну конвертацію об'єктів SQLAlchemy ORM у формат JSON.

2. **Асинхронний REST API та маршрутизація (api.py):**

Створено повноцінний FastAPI-застосунок з метаданими OpenAPI. Реалізовано повний набір ендпоїнтів для управління клієнтами та рахунками (GET, POST), а також виконання транзакційних фінансових операцій (депозити та зняття коштів) із коректною обробкою кодів станів HTTP (201 Created, 400 Bad Request, 404 Not Found).

3. **Dependency Injection та управління сесіями:**

Налаштовано механізм впровадження залежностей через функцію get_session(), що гарантує безпечне відкриття, використання та закриття сесій бази даних SessionLocal для кожного окремого HTTP-запиту.

4. **Асинхронність, фонові задачі та взаємодія із зовнішніми сервісами (external_api.py):**

Реалізовано асинхронні корутини (async def / await), пакетний ендпоїнт /accounts/batch-fetch для паралельного опитування рахунків за допомогою asyncio.gather(), а також механізми контролю мережевої стабільності: налаштування таймаутів через asyncio.wait_for (з обробкою 504 Gateway Timeout) та обмеження паралельності за допомогою asyncio.Semaphore.

5. **Тестування API (tests/test_api.py та conftest.py):**

За допомогою фреймворку pytest, pytest-asyncio та TestClient із пакету FastAPI/Starlette реалізовано набір інтеграційних тестів для перевірки роботи HTTP-маршрутів та валідації вхідних даних у реальному часі.

Результат: Успішно створено, інтегровано та протестовано асинхронне REST API банківської системи. Усі тести (pytest -v) проходять успішно (детальні результати наведено в експериментальній частині), що підтверджує готовність проєкту до захисту лабораторної роботи №8.

Експериментальна частина

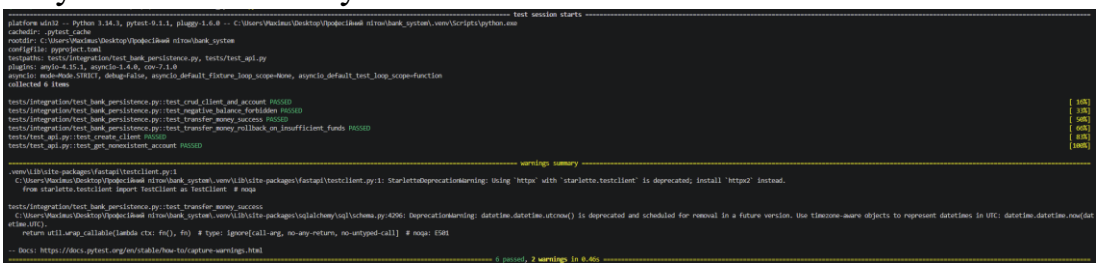
1. Результати автоматизованого тестування (pytest)

Для перевірки коректності як шару збереження даних (persistence layer), так і нових REST API ендпоїнтів, було запущено комплексний набір інтеграційних та API-тестів за допомогою фреймворку pytest. Тестування охоплює перевірку CRUD-операцій, транзакцій та HTTP-маршрутів.

Команда запуску в терміналі:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> pytest -v
```

Результат виконання у консолі:



Аналіз результатів: Усі 6 тестів (4 інтеграційні перевірки бази даних та 2 API-тести через TestClient) успішно пройшли перевірку (6 passed) без жодних помилок. Це підтверджує стабільність зв'язки FastAPI, SQLAlchemy та Pydantic-валідації.

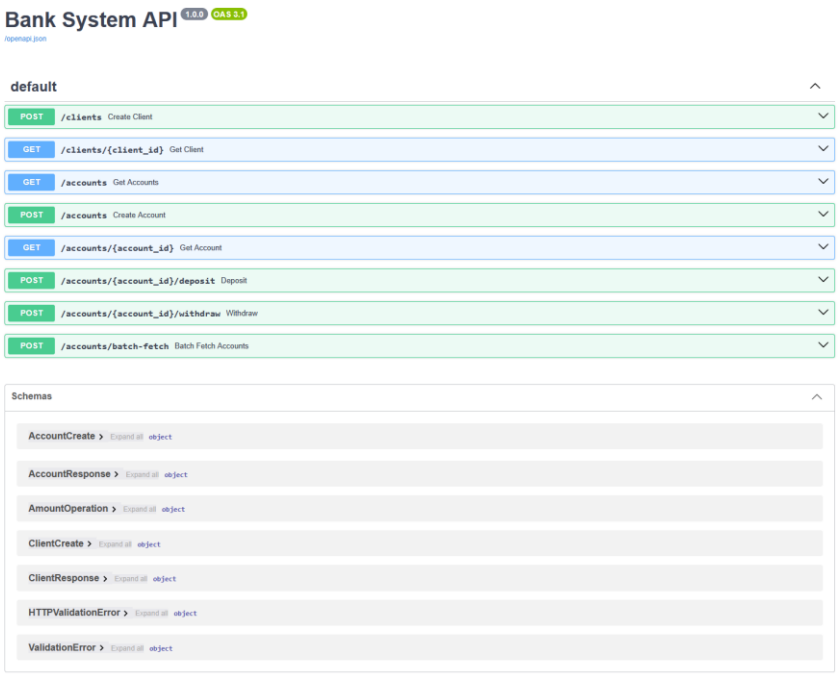
2. Інтерактивна документація (Swagger UI) та перевірка Endpoints

Після запуску локального сервера за допомогою команди:

```
(.venv) PS C:\Users\Maximus\Desktop\Професійний пітон\bank_system> uvicorn bank_system.api:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\Maximus\\Desktop\\Професійний пітон\\bank_system']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [9220] using StatReload
INFO:     Started server process [6032]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

було здійснено перехід у браузер за адресою інтерактивної документації OpenAPI: http://127.0.0.1:8000/docs.

Перевірка роботи у Swagger UI: Інтерфейс успішно відображає всі зареєстровані маршрути (/clients, /accounts, операції депозиту/зняття та пакетні запити), дозволяючи виконувати тестові запити в реальному часі зі строгим дотриманням Pydantic-схем.



Таблиця виконання вимог

№	Вимога лабораторної роботи №8	Статус виконання	Реалізація у проєкті (Варіант 4)
1	Встановлення FastAPI та Uvicorn	Виконано	Додано залежності та успішно запущено локальний сервер (uvicorn bank_system.api:app).
2	Встановлення HTTPX та pytest-asyncio	Виконано	Налаштовано середовище для асинхронних HTTP-запитів та тестування.
3	Створення schemas.py та Pydantic моделей	Виконано	Описано класи валідації (ClientCreate, AccountCreate, AmountOperation, тощо) із Field.
4	Створення api.py та FastAPI application	Виконано	Ініціалізовано додаток app = FastAPI(title="Bank System API", version="1.0.0").
5	Реалізація GET ендпоїнтів	Виконано	Реалізовано отримання даних клієнтів та рахунків за ID та загальні списки.
6	Реалізація POST ендпоїнтів	Виконано	Реалізовано створення клієнтів, рахунків та проведення фінансових транзакцій.
7	Додавання path та query parameters	Виконано	Використано параметри в шляхах ({client_id}, {account_id}) та фільтрацію в запитах.
8	Додавання валідації та response models	Виконано	Застосовано строгі правила перевірки вхідних даних та серіалізацію через response_model.

№	Вимога лабораторної роботи №8	Статус виконання	Реалізація у проєкті (Варіант 4)
9	Використання HTTP status codes	Виконано	Налаштовано повернення статусів 201 Created, 400 Bad Request, 404 Not Found, тощо.
10	Підключення Repository Layer та DI	Виконано	Інтегровано репозиторії через механізм Dependency Injection (Depends(get_session)).
11	Перевірка API через Swagger UI	Виконано	Успішно перевірено роботу ендпоїнтів через інтерактивну документацію (/docs).
12	Створення асинхронних функцій та await	Виконано	Усі ендпоїнти та сервісні виклики реалізовано як неблокуючі корутини (async def).
13	Використання asyncio.Task та gather	Виконано	Реалізовано пакетний ендпоїнт /accounts/batch-fetch для паралельного виконання завдань.
14	Реалізація timeout та retry	Виконано	Налаштовано контроль таймаутів через asyncio.wait_for із обробкою винятків.
15	Додавання Semaphore	Виконано	Застосовано семафори для захисту системних ресурсів від надмірного паралелізму.
16	Реалізація HTTPX викликів	Виконано	Використано бібліотеку HTTPX для асинхронної взаємодії із зовнішніми сервісами.

№	Вимога лабораторної роботи №8	Статус виконання	Реалізація у проєкті (Варіант 4)
17	Автоматизовані API та інтеграційні тести	Виконано	Створено тестовий набір у tests/test_api.py із використанням TestClient.
18	Запуск та перевірка через pytest	Виконано	Успішно запущено тести через pytest -v, результат: 6 passed.
19	Оновлення файлу README.md	Виконано	Додано актуальну інструкцію з встановлення, запуску сервера та тестів.

Контрольні запитання

1. **Що таке комп'ютерна мережа?**

Це сукупність комп'ютерів та інших пристроїв, з'єднаних між собою за допомогою каналів зв'язку для обміну даними та спільного використання ресурсів.

2. **Що таке клієнт-серверна архітектура?**

Це модель обчислювальної мережі, у якій задачі розділені між постачальниками послуг (серверами) та їх споживачами (клієнтами, наприклад, браузерами чи мобільними додатками).

3. **Що таке HTTP?**

HTTP (HyperText Transfer Protocol) — це протокол прикладного рівня для передачі гіпертексту, який використовується для обміну даними між клієнтом і сервером у мережі Інтернет.

4. **Що містить HTTP request?**

Він містить стартовий рядок (метод, шлях, версію протоколу), заголовки (headers) з метаданими та опціонально тіло запиту (request body).

5. **Що містить HTTP response?**

Він містить рядок стану (версію протоколу, числовий статус-код і текстовий опис), заголовки відповіді та тіло відповіді (response body) з переданими даними.

6. **Для чого використовується GET?**

Для запиту та отримання даних з сервера без зміни їхнього стану (безпечний та ідемпотентний метод).

7. **Для чого використовується POST?**

Для створення нового ресурсу або відправки даних на сервер на обробку.

8. **Для чого використовується PUT?**

Для повного оновлення або створення ресурсу за вказаною адресою.

9. **Для чого використовується PATCH?**

Для часткового оновлення існуючого ресурсу.

10. **Для чого використовується DELETE?**

Для видалення вказаного ресурсу на сервері.

11. **Що означає status code 200?**

Успішний запит (OK) — сервер успішно обробив запит і повернув результат.

12. **Що означає 201?**

Успішно створено (Created) — запит призвів до створення нового ресурсу (часто використовується після POST).

13. **Що означає 204?**

Успішно, без вмісту (No Content) — сервер успішно обробив запит, але у відповіді немає тіла (наприклад, після успішного DELETE).

14. **Що означає 404?**

Не знайдено (Not Found) — запитуваний ресурс не існує на сервері.

15. **Що означає 422?**

Необроблювана сутність (Unprocessable Entity) — синтаксис запиту правильний, але містить семантичні помилки або не проходить валідацію Pydantic.

16. **Що таке JSON?**

JSON (JavaScript Object Notation) — це легкий текстовий формат обміну даними, заснований на синтаксисі об'єктів JavaScript.

17. **Що таке REST?**

REST (Representational State Transfer) — це архітектурний стиль побудови розподілених систем, що базується на використанні стандартних методів HTTP та stateless-взаємодії.

18. **Що таке resource у REST?**

Це будь-який об'єкт чи сутність із власним станом, до якого можна отримати доступ або який можна змінити за допомогою унікального ідентифікатора (URI).

19. **Що таке endpoint?**

Це конкретна точка входу (URL-адреса разом із методом HTTP), через яку клієнт може взаємодіяти з API.

20. **Що таке path parameter?**

Це змінна частина шляху URL (наприклад, /clients/{client_id}), яка визначає конкретний ресурс.

21. **Що таке query parameter?**

Це додаткові параметри у вигляді «ключ=значення», які передаються після знака питання в URL (наприклад, ?skip=0&limit=10) для фільтрації чи пагінації.

22. **Що таке request body?**

Це дані, які клієнт надсилає на сервер у тілі HTTP-запиту (зазвичай у форматі JSON).

23. **Що таке FastAPI?**

Це сучасний, швидкий (високопродуктивний) веб-фреймворк для створення RESTful API на Python 3.8+, що використовує підказки типів (Type Hints).

24. **Для чого використовується Uvicorn?**

Це блискавично швидкий ASGI-сервер, призначений для запуску асинхронних додатків на Python (зокрема, FastAPI).

25.**Для чого потрібний Swagger UI?**

Це інтерактивний веб-інтерфейс автодокументування (на основі специфікації OpenAPI), який дозволяє тестувати ендпоїнти прямо у браузері.

26.**Що таке Pydantic?**

Це бібліотека для перевірки (валідації) та парсингу даних у Python на основі анотацій типів.

27.**Що таке validation?**

Це процес перевірки вхідних або вихідних даних на відповідність заданим правилам, типам та обмеженням.

28.**Що таке response model?**

Це схема (на базі Pydantic у FastAPI), яка визначає структуру, типізацію та фільтрацію полів даних, що повертаються клієнту.

29.**Що таке synchronous programming?**

Це модель виконання, де задачі виконуються послідовно: наступна операція блокує виконання до завершення попередньої.

30.**Що таке asynchronous programming?**

Це модель програмування, що дозволяє виконувати неблокуючі операції введення-виведення (I/O), перемикаючись між задачами під час очікування відповіді.

31.**Для чого використовується async?**

Для оголошення асинхронної функції (корутини), яка може виконуватися в асинхронному середовищі.

32.**Для чого використовується await?**

Для зупинки виконання корутини до моменту завершення асинхронної операції та отримання її результату без блокування потоку.

33.**Що таке coroutine?**

Це спеціальна функція (або об'єкт), виконання якої можна призупинити та відновити пізніше, що є основою асинхронності в Python.

34.**Що таке event loop?**

Це цикл подій — ядро асинхронного середовища, яке керує виконанням корутин, розподіляє завдання та обробляє події вводу-виводу.

35.**Що таке Task?**

Це об'єкт-обгортка в asyncio, призначений для планування та паралельного виконання корутин у фоновому режимі.

36.**Для чого використовується asyncio.create_task()?**

Для запуску корутини як паралельної фонової задачі в циклі подій.

37.**Для чого використовується asyncio.gather()?**

Для одночасного запуску кількох корутин та очікування завершення всіх їх результатів об'єднаним списком.

38.**Що таке timeout?**

Це максимальний час очікування завершення певної операції (наприклад, мережевого запиту); при його перевищенні викликається помилка.

39.**Для чого потрібний retry?**

Для автоматичного повторного виконання невдалих операцій у разі тимчасових мережевих чи серверних збоїв.

40.**Що таке backoff?**

Це стратегія збільшення затримки між повторними спробами (retry) запиту (часто експоненційна), щоб не перевантажувати сервер.

41.**Для чого використовується Semaphore?**

Для обмеження кількості паралельних завдань або потоків, які можуть одночасно отримати доступ до певного ресурсу.

42.**Що таке HTTPX?**

Це сучасний асинхронний HTTP-клієнт для Python, що підтримує як синхронні, так і асинхронні запити (HTTP/1.1 та HTTP/2).

43.**Чим httpx.AsyncClient відрізняється від синхронного клієнта?**

AsyncClient працює в асинхронному режимі (async/await), дозволяючи виконувати неблокуючі мережеві запити без зупинки інших завдань.

44.**Чому AsyncClient бажано повторно використовувати?**

Щоб підтримувати спільне з'єднання (connection pooling), зменшувати накладні витрати на встановлення TCP-з'єднань та економити ресурси.

45.**Навіщо обмежувати паралелізм?**

Щоб запобігти вичерпаню системних ресурсів (пам'яті, дескрипторів файлів, сокетів) та уникнути перевантаження цільових серверів.

46.**Чому не слід запускати тисячі tasks одночасно?**

Це може призвести до утворення «пляшкових горлечок», надмірного споживання оперативної пам'яті та помилок вичерпання лімітів з'єднань.

47.**Як REST API взаємодіє з Repository Layer?**

Ендпоїнти FastAPI викликають методи класів репозиторіїв (передаючи сесію БД), які інкапсулюють всю логіку запитів до бази даних через SQLAlchemy.

48.**Як FastAPI працює з Pydantic?**

FastAPI автоматично використовує Pydantic-схеми для валідації вхідних JSON-даних у тілі запиту, перевірки query/path параметрів та серіалізації відповідей.

49.**Як протестувати REST API?**

За допомогою бібліотеки pytest спільно з TestClient із пакету FastAPI/Starlette, який дозволяє надсилати віртуальні HTTP-запити до додатку без запуску реального сервера.

50.**Чим API integration test відрізняється від unit test?**

Юніт-тест перевіряє окрему ізольовану функцію чи метод (часто з моками), тоді як інтеграційний API-тест перевіряє взаємодію кількох шарів системи (маршрутизація, валідація, репозиторії та база даних).

Висновок: Під час виконання лабораторної роботи №8 було розроблено асинхронний RESTful API для системи банківських рахунків із використанням

FastAPI. Було реалізовано валідацію даних за допомогою Pydantic, CRUD-операції, Dependency Injection, асинхронні запити та обробку помилок. Також застосовано HTTPX для роботи із зовнішніми запитами та реалізовано автоматизоване тестування за допомогою pytest. Усі 6 тестів успішно пройшли.
У результаті роботи було отримано практичні навички розробки асинхронних вебсервісів на Python та їх автоматизованого тестування.